

PrepLab — Session 4 Notes

Network Security Controls — Testing Whether Separation Holds

Objective

Session 3 established what PrepLab's minimal controls can and cannot see. Session 4 goes further: rather than trusting the frontend/backend network diagram, every container-to-container path is actively tested — connecting from one zone and attempting to reach another — to confirm whether the intended segmentation actually holds under active probing.

Method

Baseline topology (confirmed in Session 3): **frontend** network holds web and files; **backend** network holds web and db. web is the only container attached to both, so it is the sole intended bridge between tiers. Six paths were tested: the two public-facing paths from the host, the one legitimate cross-network path, the two paths that should be blocked between files and db, and the one path that should be blocked but is not — the host-to-db path via the published port.

Test Matrix & Results

From	To	Expected	Actual	Result
Host	web	Reachable	200 OK	As designed
Host	files	Reachable	200 OK	As designed
Host	db	Blocked	Port open, TCP connects	Segmentation failure
files	db	Blocked	DNS resolution failed ("bad address")	As designed
db	files	Blocked	DNS resolution failed ("bad address")	As designed
web	db	Reachable (legitimate bridge)	Connected successfully	As designed

Step 1 — Confirm the Boundary Between files and db (Should Fail)

The most exposed, unauthenticated container (files) should have no path to the database at all — they are not even on the same Docker network.

```
$ docker exec -it preplab-files sh -c "wget -q0- --timeout=3 http://db:5432 || echo 'FAILED - db not reachable'"
wget: bad address 'db:5432'
FAILED - db not reachable from files
```

```
$ docker exec -it preplab-db sh -c "wget -q0- --timeout=3 http://files:80 || echo 'FAILED - files not reachable'"
wget: bad address 'files:80'
FAILED - files not reachable from db
```

CONFIRMED

Both directions failed with a DNS resolution error ("bad address"), not merely a blocked connection. This is a stronger result than expected: files never even learns db exists, because Docker's internal DNS only resolves

container names within the same network. This boundary is solid.

Step 2 — Confirm the Legitimate Bridge (web → db, Should Succeed)

web is the only container on both networks and is meant to reach db directly. Tested using Python's built-in socket library (wget is unavailable in this image):

```
$ docker exec -it prelab-web sh -c "python3 -c \"import socket; s=socket.socket(); s.settimeout(3); s.connect('db:5432'); print('CONNECTED to db:5432')\""
```

CONFIRMED

web reaches db exactly as intended — the one legitimate cross-network path works correctly.

Step 3 — Confirm the Public-Facing Paths (Host → web / files, Should Succeed)

```
$ curl -s -o /dev/null -w "web: %{http_code}\n" http://localhost:8080/health
$ curl -s -o /dev/null -w "files: %{http_code}\n" http://localhost:8081/
web: 200
files: 200
```

CONFIRMED

Both public-facing services respond correctly from the host, as intended.

Step 4 — The One Path That Should Be Blocked but Is Not (Host → db)

```
$ (echo > /dev/tcp/localhost/5432) 2>&1 && echo "PORT OPEN" || echo "PORT CLOSED/FILTERED"
PORT OPEN
```

SEGMENTATION FAILURE

Unlike the files↔db tests, this connection succeeds at the raw TCP level directly from the host — no Docker network membership required at all. The cause is not a broken network boundary (Steps 1–3 prove the Docker network layer itself works correctly); it is docker-compose.yml publishing the database port straight to the host (ports: ["5432:5432"]), a path that exists entirely outside the frontend/backend network model.

Conclusion

The Docker-level network segmentation is **correctly implemented**. Both blocked paths failed at the DNS level (stronger than a simple connection refusal), and the one intended bridge (web) works exactly as designed. The separation holds everywhere it is supposed to, with a single avoidable exception: the database port is published directly to the host, bypassing the entire network model through a path that sits outside it entirely — not because segmentation failed, but because one docker-compose line circumvents it. Combined with the Session 3 finding that connections to db generate no log entries under the default configuration, this exception is also invisible: an attacker using this path would leave no trace anywhere in the stack. The fix is a one-line remediation — removing the ports: mapping for db, since web already reaches it over the backend network without any host-level exposure.

Appendix — Evidence Screenshots

Raw terminal output supporting the test matrix above.

```
"Containers": {
  "342a25122f6e695033f5063e717275de985c961c589a457274b34d2712fd9976": {
    "Name": "preplab-web",
    "EndpointID": "bf5de4126cd0c96c4775a460b415e0e4c28288a404da8c60dc8f0f512994ad75",
    "MacAddress": "86:90:b6:5e:f0:93",
    "IPv4Address": "172.19.0.3/16",
    "IPv6Address": ""
  },
  "6ce90fadfa2adebba951f46247950dbf9dac6e4b9ff808980b091811a624de89": {
    "Name": "preplab-files",
    "EndpointID": "074be656d9434369e176644cb36f87187de5053e1999f49c0387d46f3755be93",
    "MacAddress": "52:60:e8:b9:af:6c",
    "IPv4Address": "172.19.0.2/16",
    "IPv6Address": ""
  }
}
```

Figure 1. docker network inspect (frontend) — confirms web and files are the only containers attached, establishing the baseline topology used for this session's tests.

```
TUF-DASH-F15@PC-ASUS MINGW64 ~/desktop/Data&App Security/Lab-20260827
$ (echo > /dev/tcp/localhost/5432) 2>&1 && echo "PORT OPEN" || echo "PORT CLOSED/FILTERED"
PORT OPEN
```

Figure 2. Direct TCP test from the host to localhost:5432 — PORT OPEN, confirming the segmentation bypass tested in Step 4.

```
TUF-DASH-F15@PC-ASUS MINGW64 ~/desktop/Data&App Security/Lab-20260827 (main)
$ docker exec -it preplab-files sh -c "wget -qO- --timeout=3 http://db:5432 || echo 'FAILED - db not reachable from files'"
wget: bad address 'db:5432'
FAILED - db not reachable from files

what's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug preplab-files
Learn more at https://docs.docker.com/go/debug-cli/

TUF-DASH-F15@PC-ASUS MINGW64 ~/desktop/Data&App Security/Lab-20260827 (main)
$ docker exec -it preplab-db sh -c "wget -qO- --timeout=3 http://files:80 || echo 'FAILED - files not reachable from db'"
wget: bad address 'files:80'
FAILED - files not reachable from db

what's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug preplab-db
Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 3. files → db and db → files — both fail with a DNS resolution error, confirming the intended network boundary holds in both directions (Step 1).

```
TUF-DASH-F15@PC-ASUS MINGW64 ~/desktop/Data&App Security/Lab-20260827 (main)
$ docker exec -it preplab-web sh -c "python3 -c \"import socket; s=socket.socket(); s.settimeout(3); s.connect(('db', 5432)); print('CONNECTED to db:5432')\""
CONNECTED to db:5432

what's next:
Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug preplab-web
Learn more at https://docs.docker.com/go/debug-cli/
```

Figure 4. web → db via Python socket connection — succeeds, confirming the one legitimate cross-network bridge works as intended (Step 2).

```
TUF-DASH-F15@PC-ASUS MINGW64 ~/desktop/Data&App Security/Lab-20260827 (main)
$ curl -s -o /dev/null -w "web: %{http_code}\n" http://localhost:8080/health
curl -s -o /dev/null -w "files: %{http_code}\n" http://localhost:8081/
web: 200
files: 200
```

Figure 5. Host → web and host → files — both return HTTP 200, confirming the public-facing paths behave correctly (Step 3).